Session 2: Orbits and Numerical Integration

Two-body motion, leapfrog versus RK4, energy conservation

astro-lab – reading notes

Goals

Understand why a symplectic integrator is the default for long gravitational simulations, and verify it by measuring energy error on an eccentric Kepler orbit.

1 The two-body problem

Two masses interact through $\mathbf{F} = -Gm_1m_2\,\hat{\mathbf{r}}/r^2$. In the centre-of-mass frame the problem reduces to one body of reduced mass μ moving in a central potential. We work in units $GM = 1$, so a test particle obeys

$$\ddot{\mathbf{r}} = -\frac{\mathbf{r}}{|\mathbf{r}|^3}.$$

Two quantities are conserved exactly:

$$E = \tfrac{1}{2}v^2 - \frac{1}{r}, \qquad \mathbf{L} = \mathbf{r} \times \mathbf{v}.$$

A numerical method is judged by how well it preserves them over many orbits.

2 Setting up an eccentric orbit

With semi-major axis $a = 1$ and eccentricity e, start at pericentre:

$$r_p = a(1-e), \qquad v_p = \sqrt{\frac{1+e}{1-e}} \quad (GM = a = 1).$$

Then $E = -1/(2a) = -1/2$ and the period is $P = 2\pi$. The script uses $e = 0.6$, which is eccentric enough to stress a fixed timestep at pericentre.

3 Two integrators

Leapfrog (kick–drift–kick)

$$\mathbf{v}_{n+1/2} = \mathbf{v}_n + \tfrac{\Delta t}{2}\mathbf{a}(\mathbf{r}_n), \quad \mathbf{r}_{n+1} = \mathbf{r}_n + \Delta t\,\mathbf{v}_{n+1/2}, \quad \mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \tfrac{\Delta t}{2}\mathbf{a}(\mathbf{r}_{n+1}).$$

It is second order, time-reversible and *symplectic*: it exactly conserves a nearby "shadow" Hamiltonian $\tilde{H} = H + O(\Delta t^2)$. So the energy error oscillates and stays bounded instead of growing.

Classical RK4

Fourth order and much more accurate per step, but not symplectic. Its energy error grows secularly because each step adds a small systematic error that does not cancel.

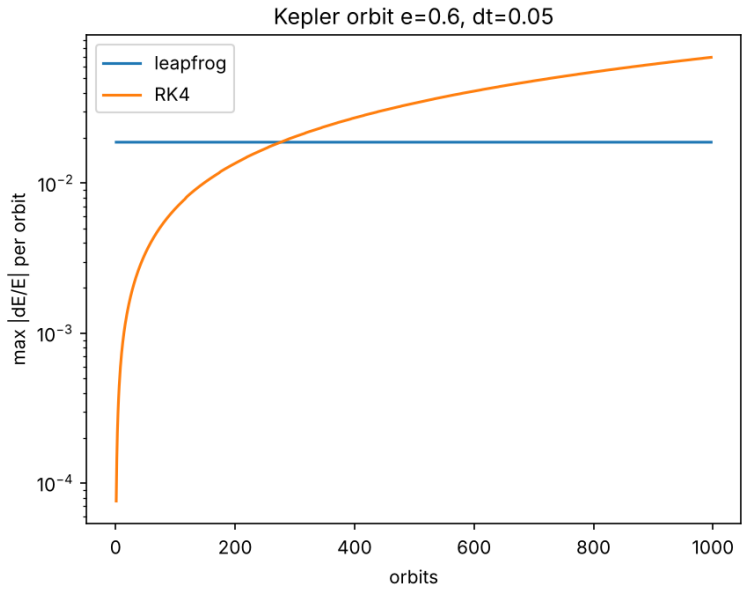
> **Key idea.** Per-step accuracy and long-term behaviour are different properties. RK4 wins at small Δt over short runs; leapfrog wins when the run is long enough for RK4's drift to accumulate.

4 Results from the repository script

Measured with `kepler_integrators.py`, $e = 0.6$:

Setting	Orbits	Leapfrog max $\lvert\Delta E/E\rvert$	RK4 max $\lvert\Delta E/E\rvert$
$\Delta t = 0.01$	100	7.4×10^{-4}	2.2×10^{-6}
$\Delta t = 0.05$	1000	1.9×10^{-2}	6.9×10^{-2}

At $\Delta t = 0.01$ RK4 is better. At $\Delta t = 0.05$ the leapfrog error stays flat at about 1.9×10^{-2}. In a separate 300-orbit run at the same Δt, RK4's error grew from 2.1×10^{-3} in the first tenth of the run to 2.0×10^{-2} in the last tenth, and by 1000 orbits it reached 6.9×10^{-2}. The figure `energy_drift.png` in the repository shows the per-orbit maximum error for the 1000-orbit run.



5 Code walkthrough

```python
"""Session 2: leapfrog vs RK4 on an eccentric Kepler orbit. Units G*M = 1.
Expect: RK4 is far more accurate per step at small dt, but its energy error grows
secularly; leapfrog's error is larger but stays bounded (symplectic), so it wins
over long runs. Try dt=0.01 vs dt=0.05 and see where the curves cross.
"""
import numpy as np
import matplotlib

```

```
matplotlib.use("Agg")
import matplotlib.pyplot as plt


def acc(r):
    return -r / np.linalg.norm(r) ** 3


def energy(r, v):
    return 0.5 * v @ v - 1.0 / np.linalg.norm(r)


def leapfrog(r, v, dt, n):
    out = np.empty(n)
    a = acc(r)
    for i in range(n):
        v = v + 0.5 * dt * a
        r = r + dt * v
        a = acc(r)
        v = v + 0.5 * dt * a
        out[i] = energy(r, v)
    return out


def rk4(r, v, dt, n):
    out = np.empty(n)
    for i in range(n):
        k1r, k1v = v, acc(r)
        k2r, k2v = v + 0.5 * dt * k1v, acc(r + 0.5 * dt * k1r)
        k3r, k3v = v + 0.5 * dt * k2v, acc(r + 0.5 * dt * k2r)
        k4r, k4v = v + dt * k3v, acc(r + dt * k3r)
        r = r + dt / 6 * (k1r + 2 * k2r + 2 * k3r + k4r)
        v = v + dt / 6 * (k1v + 2 * k2v + 2 * k3v + k4v)
        out[i] = energy(r, v)
    return out


if __name__ == "__main__":
    e = 0.6
    r0 = np.array([1 - e, 0.0])  # pericentre, a = 1
    v0 = np.array([0.0, np.sqrt((1 + e) / (1 - e))])
    E0 = energy(r0, v0)
    dt, orbits = 0.05, 1000  # coarse dt on purpose: RK4 is more accurate per
        step but drifts
    n = int(orbits * 2 * np.pi / dt)

    lf = np.abs((leapfrog(r0, v0, dt, n) - E0) / E0)
    rk = np.abs((rk4(r0, v0, dt, n) - E0) / E0)
    print(f"max |dE/E| after {orbits} orbits: leapfrog={lf.max():.2e}  rk4={rk.
        max():.2e}")

    # per-orbit maximum, otherwise the oscillating leapfrog error fills the plot
    w = int(round(2 * np.pi / dt))
    m = n // w
    env = lambda x: x[: m * w].reshape(m, w).max(axis=1)
    orbit_no = np.arange(1, m + 1)
    plt.semilogy(orbit_no, env(lf), label="leapfrog")
    plt.semilogy(orbit_no, env(rk), label="RK4")
    plt.xlabel("orbits")
    plt.ylabel("max |dE/E| per orbit")
    plt.legend()
    plt.title(f"Kepler orbit e={e}, dt={dt}")
    import pathlib
```

```
70    plt.savefig(pathlib.Path(__file__).with_name("energy_drift.png"), dpi=150,
          bbox_inches="tight")
```

6 Exercises

1. Find the Δt at which the two curves cross after 1000 orbits. How does it change with e?
2. Measure $|\mathbf{L} - \mathbf{L}_0|$ for both methods. Leapfrog conserves it to round-off for any central force: show why (the kick changes $\mathbf{v}$ by a vector parallel to $\mathbf{r}$, and the drift changes $\mathbf{r}$ by a vector parallel to $\mathbf{v}$). RK4 does not; explain what you see.
3. Implement the fourth-order Yoshida composition of leapfrog and compare with RK4.
4. Replace the fixed step with $\Delta t \propto r^{3/2}$ and describe what happens to leapfrog's energy error. (Hint: a variable step breaks symplecticity unless done carefully.)
5. Run the same setup with REBOUND (`WHFast` and `IAS15`) and compare.

References

- Hairer, Lubich, Wanner, *Geometric Numerical Integration*, ch. I and IX
- Rein & Liu (2012), REBOUND, *A&A* 537, A128
- Rein & Tamayo (2015), WHFast, *MNRAS* 452, 376